Agentic Fraud Detection via Dual-Threshold ML Classification and Generative Data Augmentation & Benchmarking

Max Raxy

Udacity Institute of AI & Technology

Capstone-7: Integrative Industry Synthesis

October 15, 2026

Agentic Fraud Detection via Dual-Threshold ML Classification and Generative Data Augmentation & Benchmarking

Contents

Overview and Industry Context **4**

Integration Rationale **4**

System Design and Technical Decisions **5**

Step-1: ML Classifier 6

Step-2: Generative Model 8

Step-3: Dual Threshold and Training Dataset for LTM 11

Ethical Considerations and Responsible AI **13**

Evaluation and Reflection **13**

Agentic System Analytics 18

Professional Relevance **19**

Future Extensions or Improvements **19**

References **19**

List of Figures

1 Agentic Workflow 6
2 Model-Comparison: Threshold Invariant Metrics. 7
3 Model-Comparison: F2-Score, Recall, Precision at model-best threshold. . . 8
4 Comparison of Generative Model, Classification-Model Pairs. 10
5 Dual Threshold Optimization, Contour Plot for Generated Data. 12
6 Dual Threshold Optimization Effect on Minimizing FP/FN. 12
7 Sample Agentic Workflow Without Escalation. 13
8 Sample Agentic Workflow Escalated to LLM. 14
9 Sample Agentic Workflow Escalated to Human. 14
10 Confusion Matrix for All Test-Case Run. 16
11 Token, Tool, Function, and Node Usage Analytics. 18

List of Tables

1 Numerical Comparison of Generative models: TVAE vs CTGAN 9
2 Test-cases Information. 15
3 Metrics for All Test-Case Run. 16

Overview and Industry Context

Credit card fraud detection is a critical challenge in the **Financial Industry**. Transactions are heavily imbalanced (e.g. less than 0.2% fraudulent cases), making precise classification vital. False Negatives (FN) incur direct financial losses, charge-back fees, and reputational damage, while False Positives (FP) create customer friction and operational review costs. Balancing high recall with precision is essential for effective automated risk mitigation.

We are using **Credit Card Fraud Detection** dataset from **Kaggle** (Machine Learning Group - ULB, 2018), which is an extremely imbalanced dataset with only 0.17% in fraud-class . The goal is to use LangGraph (LangChain Inc., 2026) and develop an agentic workflow (Plaat et al., 2025; Hughes et al., 2025) to classify the transaction to Fraud and Not-Fraud (Legit).

Integration Rationale

This project synthesizes three foundational capabilities (*ML + GenAI + Agentic*), based on three previous projects :

1. **Capstone-3:** Applied Machine Learning (Credit Card Fraud Detection)
    - In Capstone-3, we avoid any oversampling and only use **cost-sensitive learning** (Ling & Sheng, 2011; Wikipedia contributors, 2025)
    - Here, Transitioning from standard cost-sensitive learning to evaluate four oversampling methods for minority-class optimization.
2. **Capstone-5:** Generative AI Applications (Anime Face Generation with VAE)
    - In Capstone-5, we developed Variational Auto-Encoder (VAE) (Kingma & Welling, 2019; Kingma & Welling, 2022) from scratch to generate new images
    - Here, Adapting tabular-VAE (TVAE) and Conditional Tabular Generative Adversarial Networks (CTGAN) (Xu et al., 2019) from image generation to

synthesize realistic tabular credit card transactions for data augmentation (oversampling) and benchmarking/testing.

3. **Capstone-6:** Agentic AI Systems(Industrial Equipment Anomaly & Incident Response Agent)

   - In Capstone-6 we did not have real-dataset, and ML-classifier at initial stage (we used statistical threshold), and no vector database for historical context.
   - Here, Upgrading prior heuristic-based anomaly response systems by integrating a trained ML classifier, vector database retrieval, to be used in Retrieval-Augmented Generation (RAG) (Wikipedia contributors, 2026e), and a multi-tiered escalation workflow.

System Design and Technical Decisions

The architecture employs an optimized dual-threshold ML classifier to handle high-certainty transactions directly, escalating ambiguous cases to a Large Language Model (LLM) (Wikipedia contributors, 2026c) ReAct (Reasoning and Acting) agent (Yao et al., 2023) using vector database (Wikipedia contributors, 2026f) historical context (RAG) saved in Long-Term Memory (LTM). Unresolved cases escalate further to human review as shown Fig. 1. The agentic system is built on top of three main parts (steps) as we will be explained in next sections.

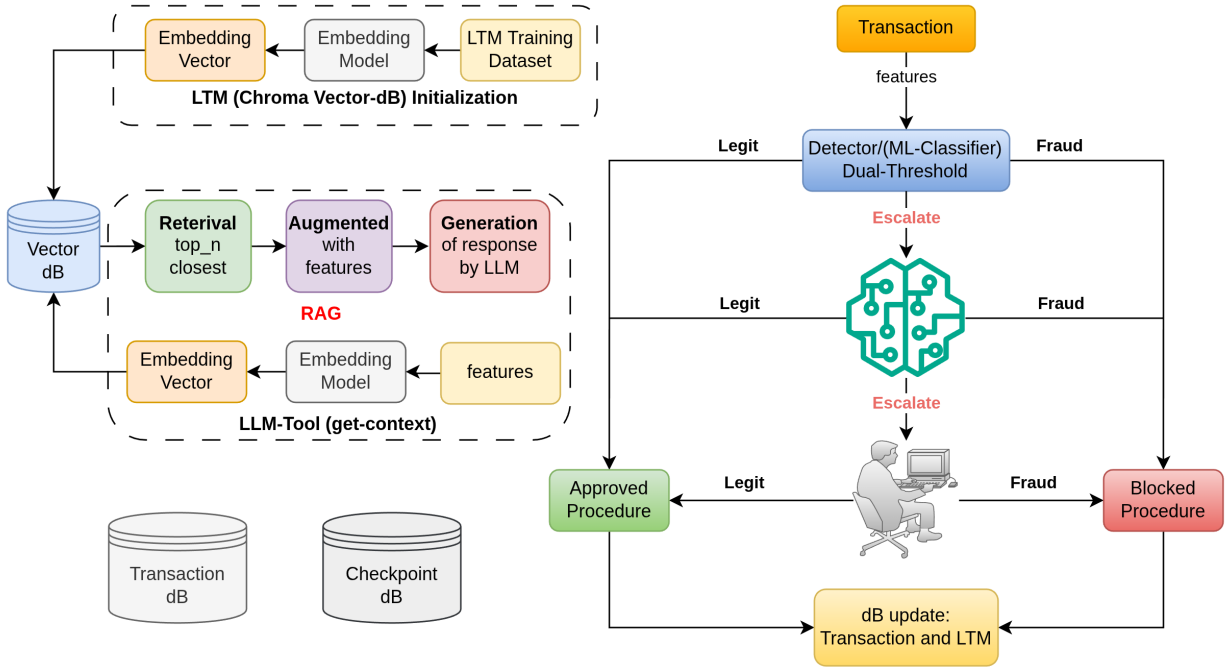


Figure 1

Agentic Workflow

Step-1: ML Classifier

The following four over-sampling methods are used and evaluated using XGBoost as the base model in all cases (Note: XGBoost shows great performance for this dataset based on Capstone-3).

1. SMOTE (Synthetic Minority Over-sampling Technique) (Chawla et al., 2002; Lemaître et al., 2017; imbalanced-learn team, 2026)

2. BSMOTE (BorderLine SMOTE) (Han et al., 2005; Lemaître et al., 2017; imbalanced-learn team, 2026)

3. TVAE (Tabular VAE) (Xu et al., 2019; Patki et al., 2016; synthetic-data-vault team, 2026)

4. CTGAN (Conditional Tabular GAN) (Xu et al., 2019; Patki et al., 2016; synthetic-data-vault team, 2026)

TVAE/CTGAN needs training and since we have small number of sample to train these generative model we limit number of epoch to avoid over-fitting (Wikipedia contributors, 2026d), and evaluate them based on final classification metrics like Precision/Recall (Wikipedia contributors, 2026a), F-Score (Wikipedia contributors, 2026b), or Precision-Recall Area Under Curve (PR-AUC) (Carrascosa, 2025) . Since in credit card fraud situation, FN can be more costly for the business and even have reputational damage consequence, we put more emphasize on Recall, and use F2-Score.

Performance comparisons are shown in Fig. 2 and Fig. 3. Model with CTGAN-augmented data is promising (PR-AUC=91.38%, and F2-Score=90.23%), and TVAE did not perform well in comparison (PR-AUC=87.06%, and F2-Score=84.42%). Also the model without oversampling (called XGB_None) perform well based on threshold invariant metrics (PR-AUC=90.12%), but when we use optimized threshold (based of F2-Score) , while it has highest precision (Precision=96.43%) , does not have good performance on recall (Recall=86.17%).

Overall while the model without oversampling and adding any artificial minority samples perform well (as also discussed in Capstone-3), but using oversampling can improve the recall (XGB_CTGAN-Recall = 90.43% vs XGB_None-Recall=86.17%) and overall F2-Score (XGB_CTGAN-F2-Score = 90.23% vs XGB_None-F2-Score=88.04%)

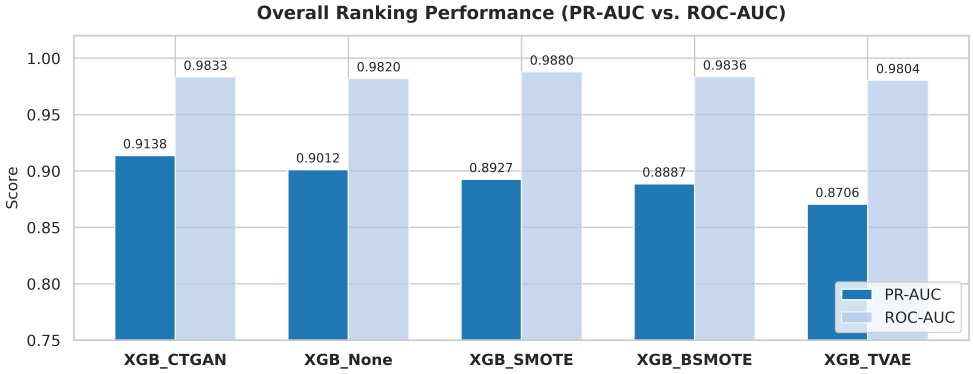


Figure 2

Model-Comparison: Threshold Invariant Metrics.

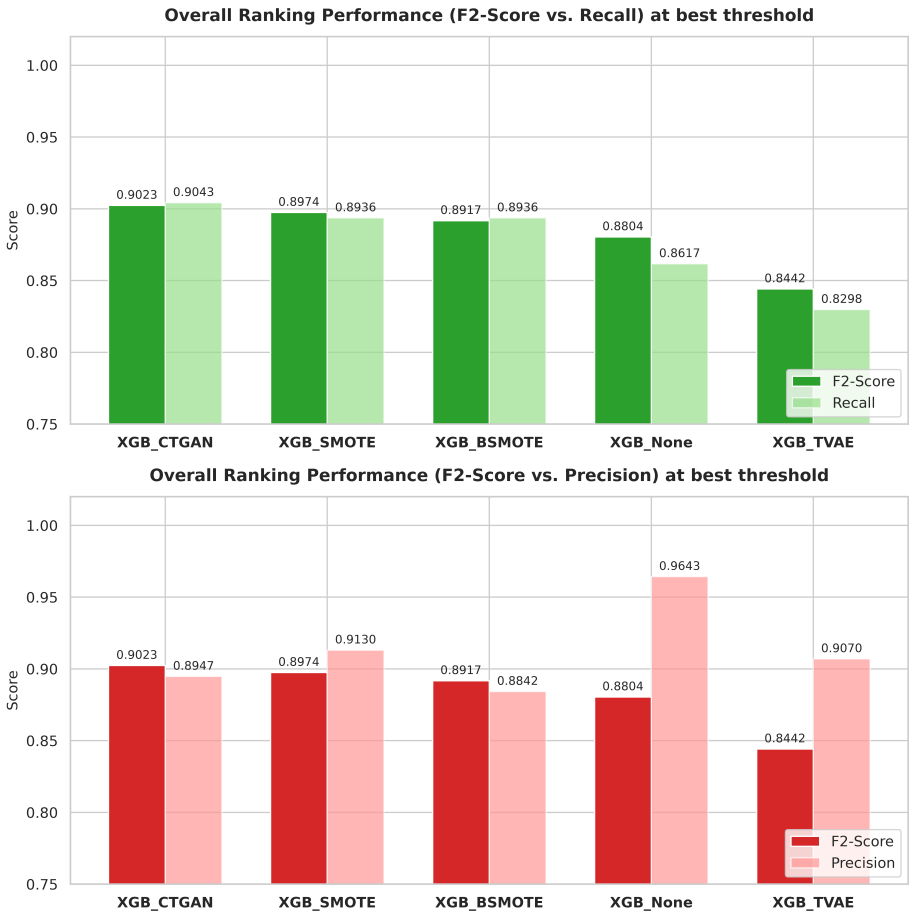


Figure 3

Model-Comparison: F2-Score, Recall, Precision at model-best threshold.

Step-2: Generative Model

We trained both TVAE and CTGAN on full dataset as our generative models to be used in testing and benchmarking our agentic systems. We check them visually using box-plot, feature-by-feature for both minority and majority class which is not easy to conclude (60-plot, each contain 3 box-plot side by side!). Then we used SDMetrics library (Synthetic data evaluation metrics) (synthetic-data-vault team, 2026), which we can be used for numerical evaluation of our generative models, and the result are given in Table 1. If we were going to judge based on these results, TVAE will be a winner, because for Fraud-class TVAE is an obvious winner both in term of overall quality score (87.51% vs

67.27%) , and number of winning features (24 vs 6), and on Legit class both TVAE/CTGAN has similar overall score of about (91%) with a little edge for CTGAN on number of winning features (17 vs 13).

Table 1

Numerical Comparison of Generative models: TVAE vs CTGAN

Class Subset	Model	Overall Quality Score	Feature Wins (out of 30)
Fraud ($Class = 1$)	**TVAE**	**87.51%**	**24 (80%)**
	CTGAN	67.27%	6 (20%)
Legit ($Class = 0$)	TVAE	91.16%	13 (43.3%)
	CTGAN	**91.62%**	**17 (56.7%)**

However, using feature by feature , and number of winning features or overall average score , cannot represent a complex relation between features and how they can determine the classification, Therefore, since our ultimate measures and metrics are PR-AUC, F2-Score, Recall, and Precision, we will use our trained classifier models in step-1, to determine these metric on the sample generated by these models, and the results are shown in Fig. 4 (NOTE: label on these plots shows (Generative-Model, Classification-Model) pair, for example CTGAN-XGB_SMOTE means we use CTGAN as Generative model, and used XGBoost model trained based on SMOTE oversampling). We also added 80% line on all plots, to be our minimum requirements on all these metrics, and help to narrow down our model-pair selection.

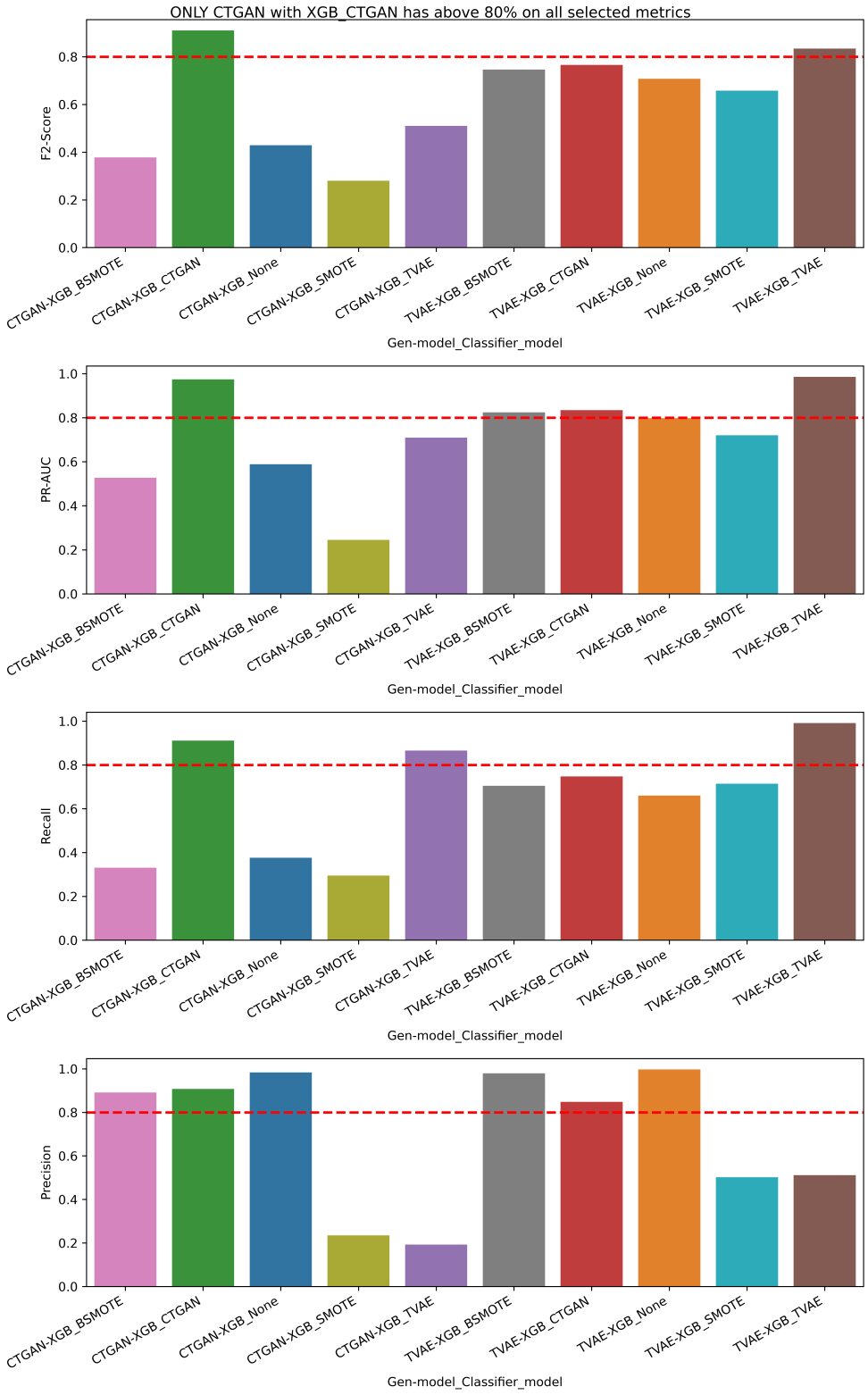


Figure 4

Comparison of Generative Model, Classification-Model Pairs.

Base on Fig. 4 we can conclude:

- Only CTGAN-XGB_CTGAN, and TVAE-XGB_TVAE have ove 80% in PR-AUC, which narrow down our selection into these two pairs.
- Between these two candidate pairs, when we check the Precision, CTGAN-XGB_CTGAN completely outperform TVAE-XGB_TVAE (approximatly 90% vs 50%)
- Data Generated by TVAE is also perform well with other classifier models, e.g: with XGB_BSMOTE, (gray-bar) XGB_CTGAN (red-bar), or even with XGB_None (orange-bar). Even better Precision with these classifier than with XGB_TVAE (brown-bar).

Step-3: Dual Threshold and Training Dataset for LTM

While for single threshold optimization we can use specific metrics, like accuracy, recall, etc, and in our case F2-Score, for dual threshold we need to define a cost function and minimize the total cost. With the same idea to punish FN twice as FP, and ***extreme simplification, to show the idea***, we define our cost function as follow:

- cost of FN: 10.0
- cost of FP: 5.0
- cost of using LLM-Agent: 0.2
- LLM-Agent accuracy: 90%
- Note: For simplicity we assume recall and precision accuracy for LLM-agent are the same.

Optimization contour-plot for generated-data shown in Fig. 5 (for real-data, please see the notebook), The effect of dual-threshold which keep FP/FN at the minimum values between single thresholds show in the confusion matrix in Fig. 6

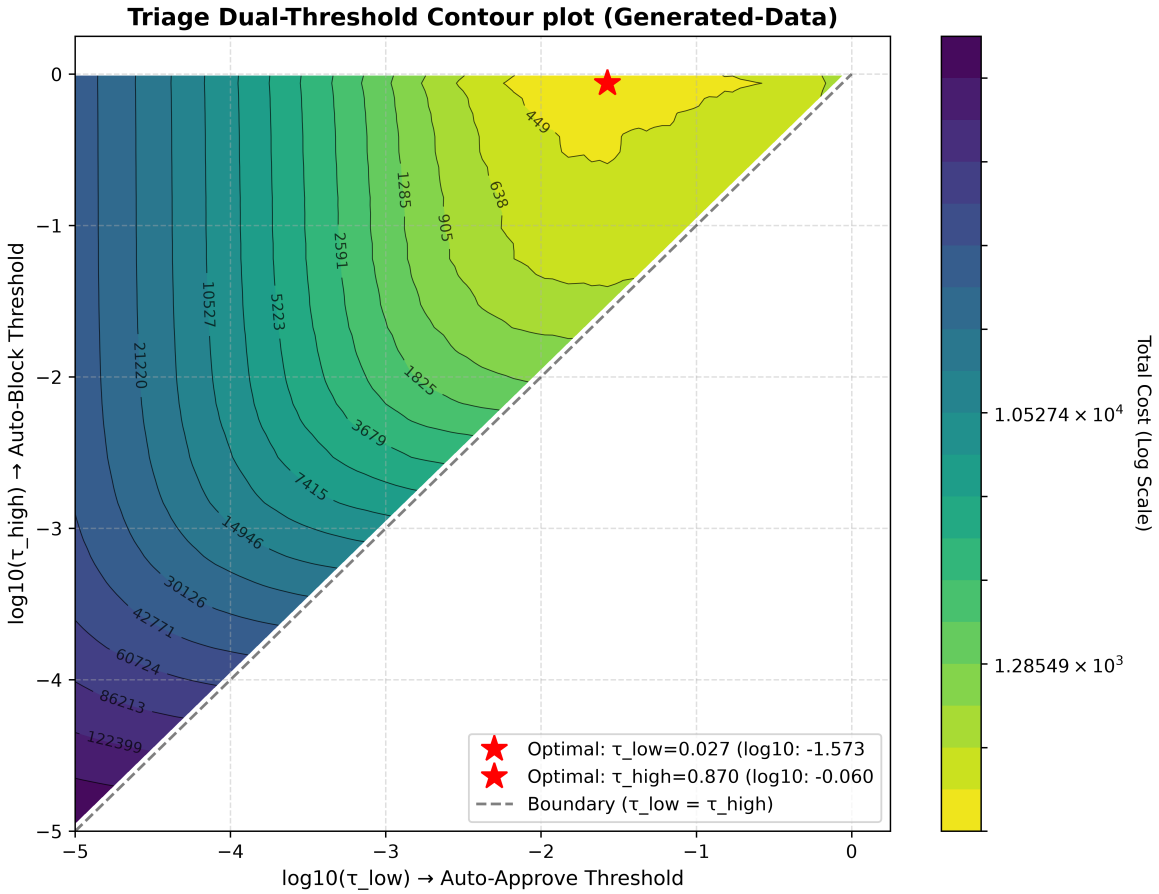


Figure 5

Dual Threshold Optimization, Contour Plot for Generated Data.

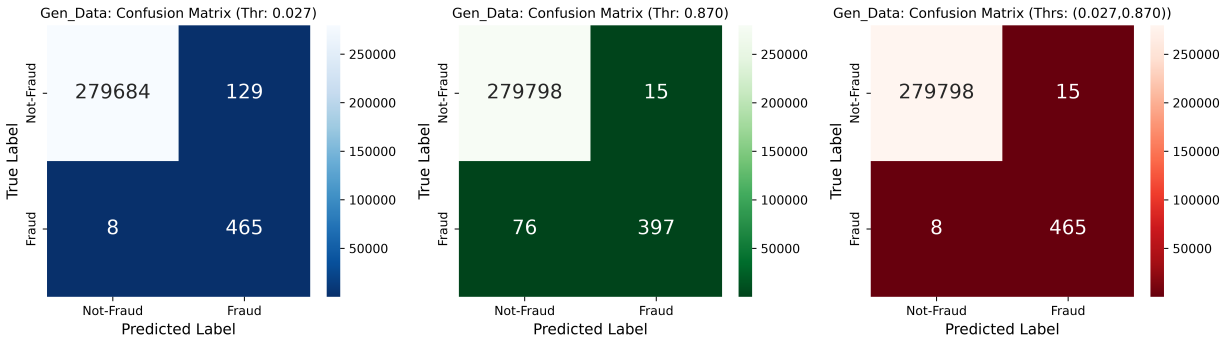


Figure 6

Dual Threshold Optimization Effect on Minimizing FP/FN.

For example Dual-Threshold-FP = min(129, 15), As we can see in Fig. 6.

With applying the optimized dual thresholds on real or generated data we can prepare samples to initialize our Long-Term Memory (Vector-database) to be used by LLM-Agent.

Ethical Considerations and Responsible AI

Automated financial screening introduces important ethical dimensions regarding fairness, transparency, and accountability:

- **Algorithmic Bias & False Positives:** Imbalanced datasets risk generating biased rules that unfairly target specific demographic spending behaviors as anomalous.
- **Human-in-the-Loop Accountability:** By incorporating mandatory human escalation for low-confidence decisions, the system prevents autonomous wrongful account blocking and ensures user due process.
- **Data Privacy:** Synthetic data generation (CTGAN) helps mitigate privacy leakage by supplementing training pipelines without over-relying on sensitive real-world cardholder records.

Evaluation and Reflection

Sample agentic-system run without escalation (Fig. 7), escalate to LLM (Fig. 8), and escalate up to Human (Fig. 9) are shown below. (For all possible scenarios including when Agent make mistake and create FP see the `step4_agentic_test_results` notebook or its html version)

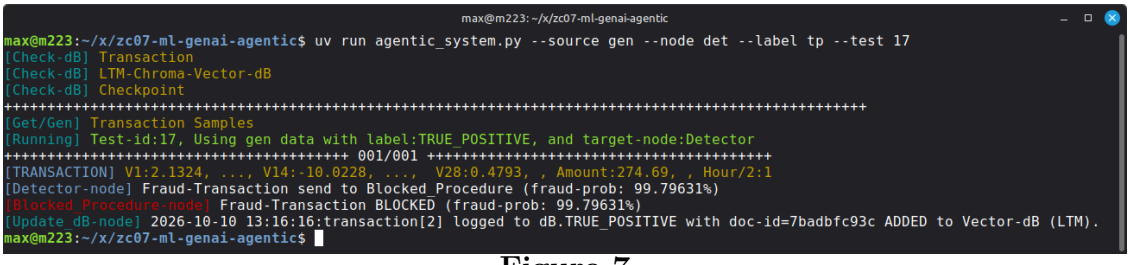


Figure 7

Sample Agentic Workflow Without Escalation.

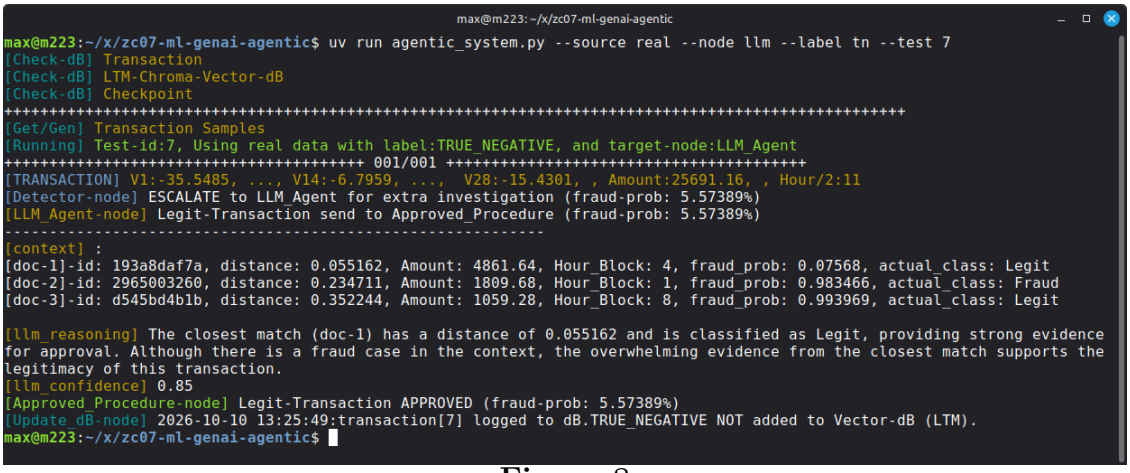


Figure 8

Sample Agentic Workflow Escalated to LLM.

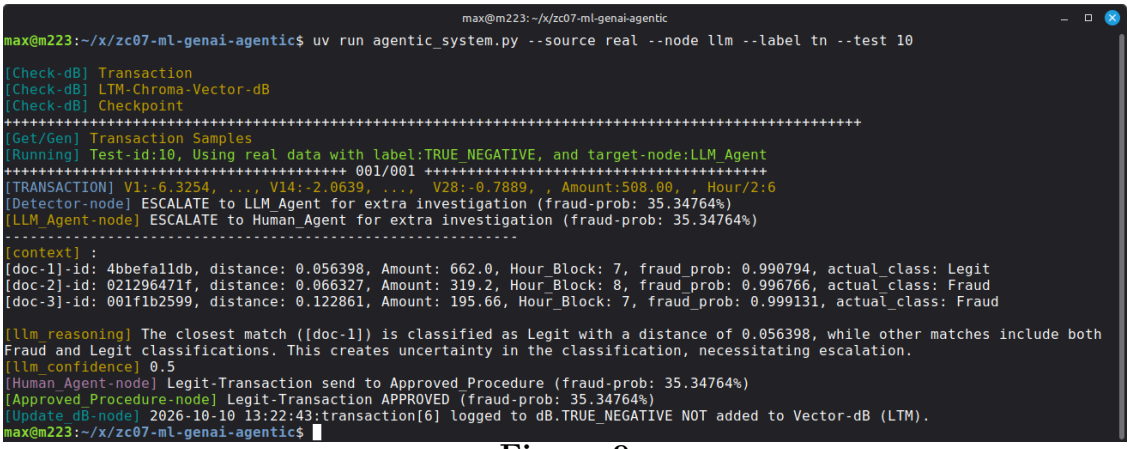


Figure 9

Sample Agentic Workflow Escalated to Human.

When we prepared data for LTM, in step-3, we split them 50%-50% into training and testing dataset, we will use training dataset for initializing LTM, and testing (test-cases) for our test. In total we have 661 test-cases which is a mixture of data from real-dataset, and generated(gen)-dataset, and targeting our Detector-Node (Dual-Threshold Classifier) or LLM-Agent Node as shown in Table 2

For detector-node we have FP/FN, because these FP/FN are the results of our ML classifier, and we cannot get rid of them unless we could train a better classifier models.

Table 2

Test-cases Information.

node_source	TP	TN	FN	FP	Row_Count
Detector-real	225	50	4	2	281
LLM_Agent-real	8	21	0	0	29
Detector-gen	199	50	4	7	260
LLM_Agent-gen	34	57	0	0	91
Column_Sum	466	178	8	9	**661**

For LLM node, there is no FP or FN sample in the above table, FP or FN will be created depends on LLM-Agent decision which in turn also depends on how good is the content of LTM, and we will see the result after test.

While real-dataset is limited, we can increase test-cases using generated data, as far as we have budget to spend for LLM cost! Therefore to limit our cost, we use limited number of generated data, too.

The first time we run all test cases, LLM has only the LTM initialized with (training cases), but if we run the test-cases again, we expect that LLM to have perfect decision, because LTM should have all the previous mistake. The confusion-matrix of the results shown in Fig. 10, and corresponding numerical metrics collected in Table 3.

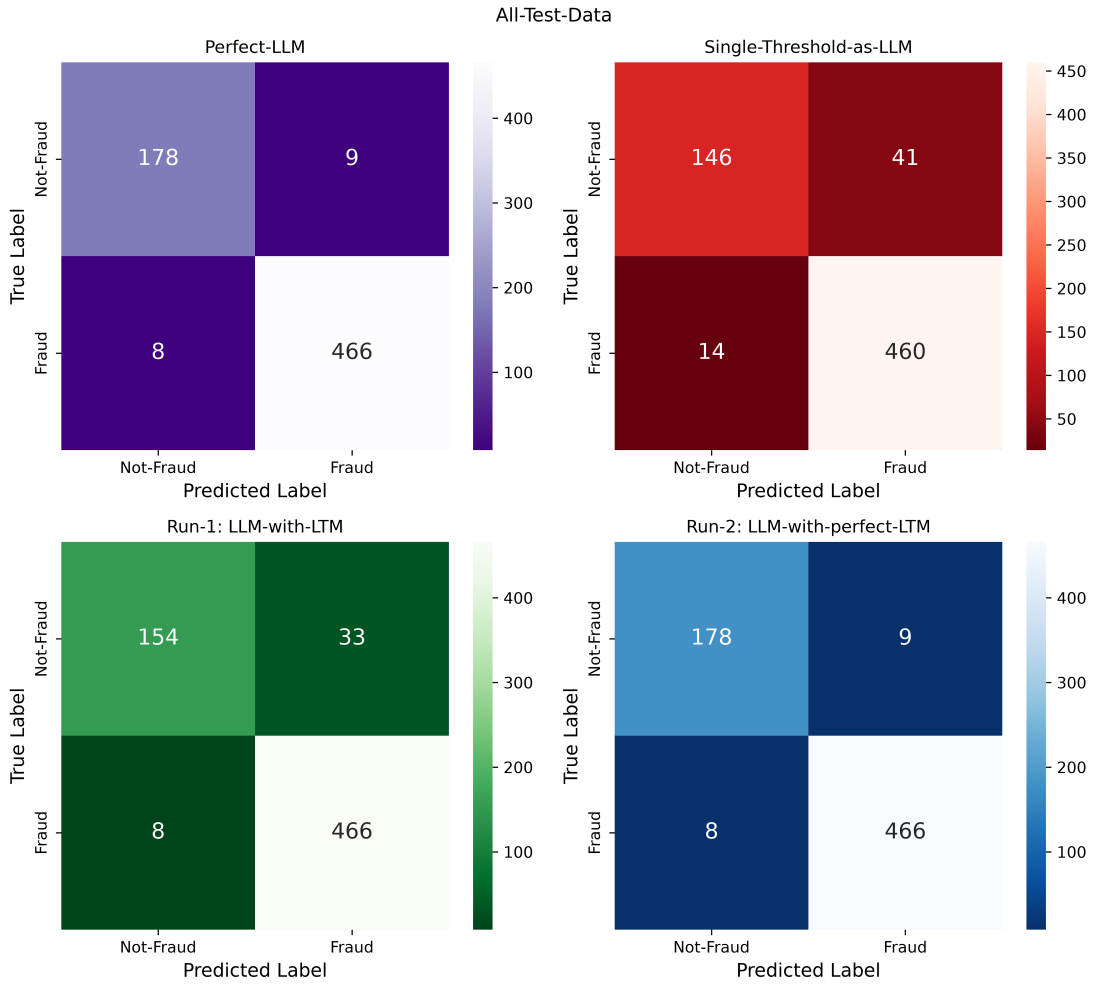


Figure 10

Confusion Matrix for All Test-Case Run.

Table 3

Metrics for All Test-Case Run.

Run-Info	Accuracy	Precision	Recall	F1-Score	F2-Score
Perfect-LLM	97.43	98.11	98.31	98.21	98.27
Single-Threshold-as-LLM	91.68	91.82	97.05	94.36	95.95
Run-1: LLM-with-LTM	93.80	93.39	98.31	95.79	97.29
Run-2: LLM-with-perfect-LTM	97.43	98.11	98.31	98.21	98.27

Comparison between **Single-Threshold-as-LLM** and **Run-1: LLM-with-LTM** shows that how using LLM with initialized LTM, avoid adding any FN (beyond what happened at detector node which is the accuracy of our Classification-Model), but single -threshold actually adds up 6 more FN, and for FP, while LLM agent add extra 24 FP, but with more mature LTM they can be reduced (as we see in Run-2 as ideal LTM), but single threshold agent add extra 32 FP, which results in 1.34% reduction in F2-Score (97.29% to 95.95%)

Based on the above result (Run-1), we can calculate agent-metrics as :

- **Accuracy** 80.00%
- **Precision** 63.64%
- **Recall** 100.00%
- **F2-Score** 89.74%

This shows that Agent really avoid FN since in the prompt we emphasize that. Also it shows that the recall/precision for agent is not the same, and after more test, we may need to update our cost function, and define different recall/precision values instead of single accuracy for agent and re-calculate the optimal dual threshold which we consider it as future work after more more tests in real production environment.

Rigorous testing across 661 cases demonstrated strong performance (F_2-Score of 97.29% with LTM). Key takeaways include:

- **Successes:** Dual thresholds successfully filtered unambiguous traffic, allowing the LLM and human agents to focus solely on high-risk edge cases. Long-Term Memory (LTM) integration markedly improved sequential accuracy.
- **Limitations:** Initial agent prompt engineering caused asymmetric precision/recall behavior compared to assumptions in the cost function, highlighting the need to decouple agent-specific precision and recall metrics during optimization.
- **Potential Metric Inflation**: Due to limited fraud samples (only 473), the same data (or portions of it) was leveraged in both step-1, and step-2. This approach was

necessary for initial feasibility, it can introduces *data leakage*, potentially leading to optimistic metrics. Let say, we have 5% performance inflation, then the realistic F2-Score is 92.2%. Solution for production discussed in last section.

Agentic System Analytics

We also use LangGraph Checkpoint database to plot our agentic workflow analytics shown in Fig. 11

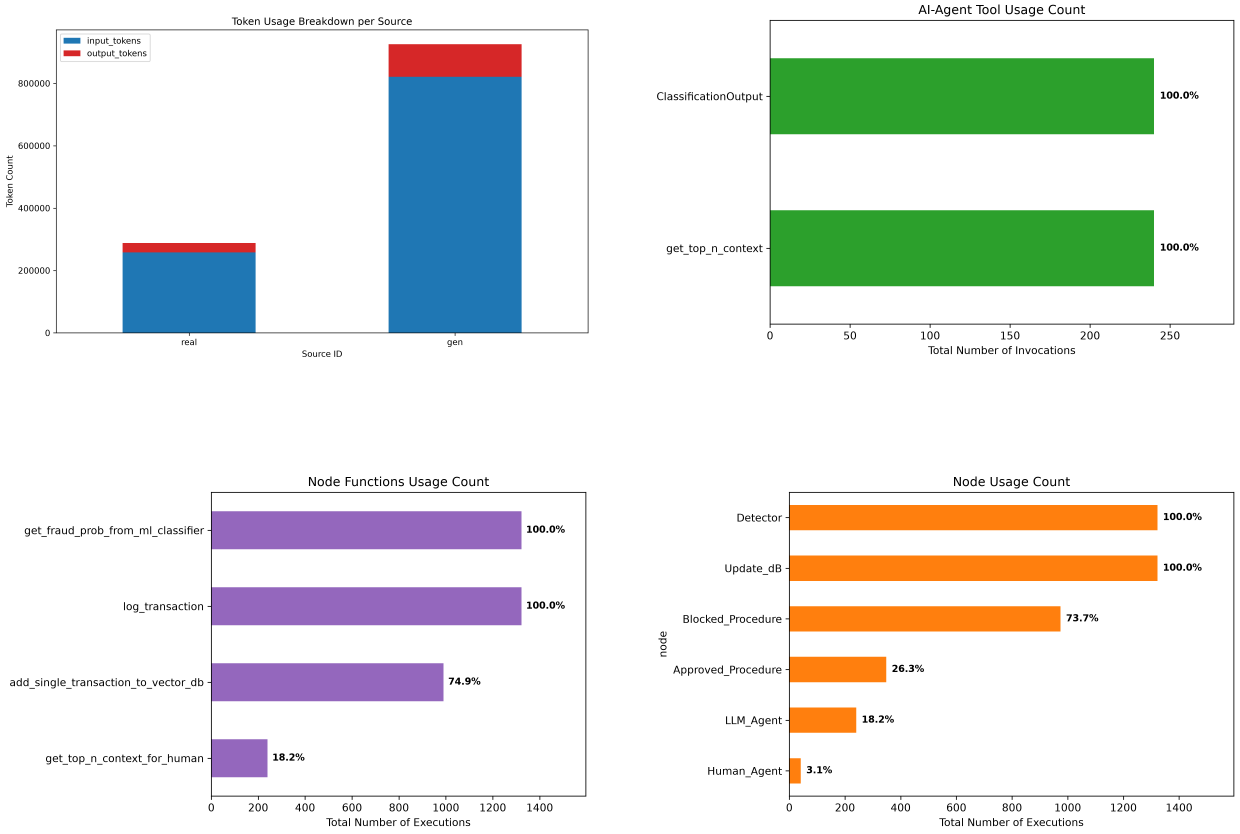


Figure 11

Token, Tool, Function, and Node Usage Analytics.

It's worth to emphasize the following regarding node-usage:

- LLM was only involved in 18.2% of all interactions
- Human involved in 3.1% of all interactions or $\frac{3.1}{18.2} \approx 17\%$ of LLM interaction.

Professional Relevance

This project demonstrates end-to-end engineering readiness for modern AI roles by successfully bridging traditional predictive modeling, generative data augmentation, and agentic orchestration frameworks (LangGraph). It highlights a practical capability to design cost-aware enterprise systems that balance automated scale with human-in-the-loop safety.

Future Extensions or Improvements

1. Refine the dual-threshold cost function by decoupling distinct agent precision and recall parameters based on production error distributions.

2. Deploy real-time feedback loops to continuously update the vector database LTM with live verified outcomes.

3. Expand automated evaluation suites to test resilience against evolving, adversarial fraud patterns.

4. To avoid possible data leakage due to usage of same data in both step-1: ML-classifier and step-2: Generative model, in production, with the assumption of at least 400-500 samples for robust training of generative model and 200 samples for final agentic test, we need at least 1200 minority samples, then we can divide dataset into 3 splits of 500 for step-1 (400 for training), 500 for step-2, and 200 for final test. Also this split need to be done properly using stratification on continuous data which has its own challenge (we may need to use quantile-based discretization prior to stratification).

References

Carrascosa, I. P. (2025, September). *Roc auc vs precision-recall for imbalanced data* [Accessed: 2026-10-15]. Machine Learning Mastery. https://machinelearningmastery.com/roc-auc-vs-precision-recall-for-imbalanced-data/

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, *16*, 321–357. https://doi.org/10.1613/jair.953

Han, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-smote: A new over-sampling method in imbalanced data sets learning. In D.-S. Huang, X.-P. Zhang, & G.-B. Huang (Eds.), *Advances in intelligent computing* (pp. 878–887). Springer Berlin Heidelberg.

Hughes, L., Dwivedi, Y. K., Malik, T., Shawosh, M., Albashrawi, M. A., Jeon, I., Dutot, V., Appanderanda, M., Crick, T., De', R., Fenwick, M., Gunaratnege, S. M., Jurcys, P., Kar, A. K., Kshetri, N., Li, K., Mutasa, S., Samothrakis, S., Wade, M., & Walton, P. (2025). Ai agents and agentic systems: A multi-expert analysis. *Journal of Computer Information Systems*, *65*(4), 489–517. https://doi.org/10.1080/08874417.2025.2483832

imbalanced-learn team. (2026). Imbalanced-learn documentation [Accessed: 2026-10-14]. https://imbalanced-learn.org/stable/index.html

Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders. *CoRR*, *abs/1906.02691*. http://arxiv.org/abs/1906.02691

Kingma, D. P., & Welling, M. (2022). Auto-encoding variational bayes. https://arxiv.org/abs/1312.6114

LangChain Inc. (2026). Langgraph overview [Accessed: 2026-09-15]. https://docs.langchain.com/oss/python/langgraph/overview

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, *18*(17), 1–5. http://jmlr.org/papers/v18/16-365.html

Ling, C. X., & Sheng, V. S. (2011). Cost-sensitive learning and the class imbalance problem. In C. Sammut & G. I. Webb (Eds.), *Encyclopedia of machine learning* (pp. 231–235). Springer. https://doi.org/10.1007/978-0-387-30164-8_181

Machine Learning Group - ULB. (2018). *Credit card fraud detection [dataset]*. Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Patki, N., Wedge, R., & Veeramachaneni, K. (2016). The synthetic data vault. *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 399–410. https://doi.org/10.1109/DSAA.2016.49

Plaat, A., Van Duijn, M., Van Stein, N., Preuss, M., Van der Putten, P., & Batenburg, K. J. (2025). Agentic large language models, a survey. *Journal of Artificial Intelligence Research*, *84*. https://doi.org/10.1613/jair.1.18675

synthetic-data-vault team. (2026). The synthetic data vault project [Accessed: 2026-10-14]. https://github.com/sdv-dev

Wikipedia contributors. (2025). Cost-sensitive machine learning — Wikipedia, the free encyclopedia [Online; accessed 14-August-2026]. https://en.wikipedia.org/w/index.php?title=Cost-sensitive_machine_learning&oldid=1297370809

Wikipedia contributors. (2026a). Evaluation of binary classifiers — Wikipedia, the free encyclopedia [Online; accessed 15-October-2026]. https://en.wikipedia.org/w/index.php?title=Evaluation_of_binary_classifiers&oldid=1339785632

Wikipedia contributors. (2026b). F-score — Wikipedia, the free encyclopedia [Online; accessed 15-October-2026]. https://en.wikipedia.org/w/index.php?title=F-score&oldid=1378835488

Wikipedia contributors. (2026c). Large language model — Wikipedia, the free encyclopedia [Online; accessed 15-September-2026]. https://en.wikipedia.org/w/index.php?title=Large_language_model&oldid=1374737380

Wikipedia contributors. (2026d). Overfitting — Wikipedia, the free encyclopedia [Online; accessed 15-October-2026]. https://en.wikipedia.org/w/index.php?title=Overfitting&oldid=1374396766

Wikipedia contributors. (2026e). Retrieval-augmented generation — Wikipedia, the free encyclopedia [Online; accessed 14-October-2026]. https://en.wikipedia.org/w/index.php?title=Retrieval-augmented_generation&oldid=1377227095

Wikipedia contributors. (2026f). Vector database — Wikipedia, the free encyclopedia [Online; accessed 14-October-2026]. https://en.wikipedia.org/w/index.php?title=Vector_database&oldid=1374243120

Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling tabular data using conditional gan. https://arxiv.org/abs/1907.00503

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629